

Lab FS 实验报告

李睿阳 524120910059

1. Subtask 实现思路与修改内容

Subtask 1: Large Files (大文件支持)

实现思路：

xv6 原有的 inode 结构包含 12 个直接索引块和 1 个一级间接索引块，最大支持 268 个 block。本任务要求增加二级间接索引块支持。

1. 将 `NDIRECT` 修改为 11，腾出一个位置给二级间接块。
2. 在 `struct dinode` 和 `struct inode` 的 `addrs` 数组中增加一个槽位，使其大小变为 `NDIRECT + 2`。
3. 修改 `bmap()` 函数：处理逻辑块号 `bn` 落在二级间接块范围内的映射逻辑。二级块存储 256 个一级间接块的地址，每个一级间接块存储 256 个数据块地址。
4. 修改 `itrunc()` 函数：确保释放文件时，递归地释放二级间接块、一级间接块及其下属的所有数据块。

修改的文件：

- [kernel/fs.h](#)
- [kernel/file.h](#)
- [kernel/fs.c](#)

关键代码：

```

// kernel/fs.c: bmap()
static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a, *a1, *a2;
    struct buf *bp, *bp1, *bp2;

    // ... 直接索引与一级间接索引代码省略 ...

    bn -= NINDIRECT;

    if (bn < NINDIRECT * NINDIRECT) {
        // 处理二级间接块
        if ((addr = ip->addrs[NINDIRECT + 1]) == 0) {
            addr = balloc(ip->dev);
            if (addr == 0) return 0;
            ip->addrs[NINDIRECT + 1] = addr;
        }
        bp1 = bread(ip->dev, addr); // 读取二级间接块
        a1 = (uint*)bp1->data;
        if ((addr = a1[bn / NINDIRECT]) == 0) {
            addr = balloc(ip->dev);
            if (addr == 0) {
                brelse(bp1);
                return 0;
            }
            a1[bn / NINDIRECT] = addr;
            log_write(bp1);
        }
        bp2 = bread(ip->dev, addr); // 读取下属的一级间接块
        a2 = (uint*)bp2->data;
        if ((addr = a2[bn % NINDIRECT]) == 0) {
            addr = balloc(ip->dev);
            if (addr) {
                a2[bn % NINDIRECT] = addr;
                log_write(bp2);
            }
        }
        brelse(bp2);
        brelse(bp1);
        return addr;
    }
}

```

```
panic("bmap: out of range");  
}
```

Subtask 2: Symbolic links (符号链接)

实现思路：

实现 `symlink(target, path)` 系统调用并在 `open()` 中处理软链接跳转。

1. 定义新的文件类型 `T_SYMLINK` 和打开标志 `O_NOFOLLOW`。
2. `sys_symlink`：创建一个类型为 `T_SYMLINK` 的新 inode，将目标路径（`target`）作为数据写入该 inode 的数据块中。
3. `sys_open`：如果打开的文件是软链接且未指定 `O_NOFOLLOW`，则循环读取该链接指向的路径并重新执行查找，直到找到非链接文件或达到最大递归深度。

修改的文件：

- [kernel/stat.h](#)
- [kernel/fcntl.h](#)
- [kernel/sysfile.c](#)
- [kernel/syscall.c](#), [kernel/syscall.h](#)
- [user/user.h](#), [user/usys.pl](#)

关键代码：

```

// kernel/sysfile.c: sys_symlink()
uint64
sys_symlink(void){
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;
    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;
    begin_op();
    if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
        end_op();
        return -1;
    }
    // 将目标路径写入 inode 数据块
    if(writei(ip, 0, (uint64)target, 0, strlen(target)) != strlen(target)){
        iunlockput(ip);
        end_op();
        return -1;
    }
    iunlockput(ip);
    end_op();
    return 0;
}

```

```

// kernel/sysfile.c: sys_open() 增加软链接处理
int depth = 0;
char target_buf[MAXPATH];
while(ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)){
    if(depth >= 10){ // 防止死锁/无限循环
        iunlockput(ip);
        end_op();
        return -1;
    }
    depth++;
    if(readi(ip, 0, (uint64)target_buf, 0, ip->size) != ip->size){
        iunlockput(ip);
        end_op();
        return -1;
    }
    target_buf[ip->size] = '\0'; // 手动加结束符
    iunlockput(ip); // 移动前必须释放当前锁
    if ((ip = namei(target_buf)) == 0){
        end_op();
        return -1;
    }
}

```

```
}  
ilock(ip);  
}
```

2. 调试中的要点（任务）

1. **清理磁盘镜像：** 修改 `NDIRECT` 等文件系统元数据宏后，必须执行 `make clean` 以重新生成 `fs.img`，否则 `mkfs` 计算的磁盘布局将不匹配。
2. **并发保证：** 在 `sys_symlink` 中创建文件和写入内容时，必须包裹在 `begin_op()` 和 `end_op()` 之间。

3. 问题与收获

- **a. log_write：** 任何对磁盘块内容的修改（如在 `bmap` 中分配新块并更新间接块内容）都必须调用 `log_write`，否则这些修改不会被提交到日志，导致系统崩溃后数据丢失或文件系统损坏。
- **b. bread 与 brelse 的搭配使用：** 每次 `bread` 获取缓冲区后，无论成功与否（只要返回了 `buf`），在不再使用时必须显式调用 `brelse` 释放对缓冲区的引用，防止缓冲区泄露导致系统卡死。
- **c. begin_op 与 end_op 的搭配使用：** 文件系统的系统调用必须严格成对使用这两个函数。这保证了事务的原子性，特别是在涉及多个磁盘写操作（如 `symlink` 的创建与写入）时，避免系统崩溃产生状态不一致。
- **d. symlink 中 iunlockput(ip) 要及时释放：** 在 `sys_symlink` 成功写入路径后，必须使用 `iunlockput` 释放 `inode` 的锁分并减少引用计数。
- **e. target_buf[ip->size] = '\0'：** `readi` 读取内容时不会自动在缓冲区末尾添加字符串结束符。在 `sys_open` 中解析软链接目标路径时，必须根据 `ip->size` 手动补齐 `\0`，否则 `namei` 可能会读取到内存中的脏数据导致解析失败。
- **f. 移动 ip 前先 unlock 原先 ip 的锁：** 在 `sys_open` 的跳转循环中，在调用 `namei` 查找下一个 `inode` 之前，必须先对当前的 `T_SYMLINK` 节点执行 `iunlockput`。否则，如果 `namei` 路径中包含当前正在锁定的 `inode`，会发生自死锁。
- **g. 设置最大递归深度：** 软链接可能形成环路（如 `ln -s a b; ln -s b a`）。通过设置深度阈值（如 10），可以在链接过多时强行报错返回，防止内核陷入无限死循环并耗尽栈空间。